Supplementary Material: Organizational Visibility in Hierarchical LLM Agents

Anonymous supplementary material · Supporting protocols and reproducibility artifacts

This supplement documents the implemented contrast, sample and scoring protocols, and supporting process analyses. Its quantitative results use the same indexed experiment records as the main paper. Additional exploratory results are separated from the primary comparisons. The package contains anonymous source manifests, frozen quantitative evidence, exact intervention blocks, and an offline verification entry point. Historical V2 labels in the records denote ORG-DESC in the paper.

S1. What the Organizational Description Supplies

The unit of disclosure is the actual model input. A role name in callback metadata, a task dependency in program state, or a supervisor's internal graph configuration does not establish that a model received that information. Our context audit distinguishes explicit disclosure, conditional disclosure through available history or results, possible inference from other text, and information not uniformly disclosed. It establishes visibility, not a model's internal comprehension or compliance.

In ROMA, the root task is at depth zero, and generated child tasks increase depth by one. Atomizer, Planner, Executor, and Aggregator are functional stages repeated across task depths. They are not four authority levels. Native task inputs can disclose current goal, overall objective, current depth, maximum depth, and whether the task is at the limit. At the depth cap, native execution can bypass further atomization and planning and execute directly. Planner contexts contain decomposition and dependency contracts; aggregators receive direct child goals and results. Completed dependency results and available artifacts may be supplied conditionally. Artifact visibility can extend beyond an immediate dependency under the configured native behavior; we therefore do not define BASE as a strict one-hop information boundary.

The global ROMA description explains these functional roles, recursive reuse, atomic execution, dependency-aware processing, and upward aggregation. Its optional Verifier entry does not activate verification or imply that the analyzed runs used a Verifier. It does not disclose a future task tree, a complete live dependency graph, every sibling, or every task's private history. Its static general flow also does not override the original maximum-depth rule.

In TalkHier Physics, the configured organization contains six roles on three organizational levels: Default Supervisor at the root; Answer Generator, Revisor, and Evaluator Supervisor as its direct members; and Physics and Mathematics below Evaluator Supervisor. A selected direct member returns to its supervisor before another member is selected. An evaluation-team invocation returns through Evaluator Supervisor to Default Supervisor. Dispatch is serial within each team, and the main team waits for the selected evaluation-team invocation. The configured implementation has no separate subtask dependency DAG or expert-to-expert dispatch interface. Internal agent/tool/respond nodes are computational phases rather than additional organizational members.

These enforced routes must be distinguished from prompt requirements. The native main-team prompt requests generation first and revision before completion, and re-evaluation instructions request all relevant experts. The graph does not implement a visit-check gate that forbids completion until those requested visits have occurred. ORG-DESC describes both the available routes and those inherited prompt requirements; it neither turns the latter into hard constraints nor adds dispatch permissions. Native local history is selected by the graph, with an initial history limit and possible subsequent tool-loop additions, rather than a full global execution transcript.

The same six-role Physics configuration is used for the College Physics implementation and this study's external MMLU-Pro Physics data adapter. Other inspected MMLU domains retain the audited management levels but differ in expert sets: Moral Scenarios has four experts and eight total roles; Machine Learning has three and seven; Formal Logic has five and nine; US Foreign Policy has three and seven. Domain-specific responsibilities, tools, and prompt requirements also vary. The fixed checkout does not provide equivalent complete runtime-context evidence for WikiQA or Camera, so paper descriptions of those tasks are not used to assert identical organizational disclosure.

The context audit checked native source construction against actual model inputs. For the MMLU-Pro batch, 2,308 BASE model starts contain no ORG-DESC prefix and 3,093 ORG-DESC starts contain the same single prefix in the first system message. Removing that prefix makes the 100 first paired requests identical in the checked inputs. Later histories can differ as the executions diverge. These start counts are audit coverage rather than counts of normal returned calls on the 94-question behavioral panel.

The architecture comparison in Figure 1 illustrates representative calls, not a measured percentage of organizational knowledge. The highlighted TalkHier roster is relative to Evaluator Supervisor; root and leaf disclosures are listed separately. ROMA task depths illustrate recursive reuse, and the drawn modules need not all run on a given task. The depth cap forces direct execution. Green description panels refer to the existing static template and never represent added dispatch edges or a complete live task tree. The code paths and representative archived inputs used for the figure are preserved in architecture_audit/.

S2. Samples, Pairing, and Outcome Evaluation

The primary SEAL-0 scope is the 94-question H/R1/R2 panel, with 272 common valid question–round pairs and 544 corresponding arm executions. For metric m, each arm is first averaged over the same valid rounds within question i; those question means are then averaged over 94 questions. Extra N rounds, older 111/011 interventions, and incomplete pilots are separate experiments and do not enter these means. Eighty-six questions have three paired rounds, six have two, and two have one. The latter two, indexed 6 and 96 in the archived panel, do not enter three-attempt reliability. Questions 38, 55, 62, and 100 lack R2; 87 and 92 lack R1. This convention preserves available repeated evidence without making a question's statistical weight proportional to its number of runs. Some SEAL-0 runs are blocked by provider safety/content policies; blocked outcomes and execution errors retain their unavailable status rather than being scored as incorrect, and matching uses only question–round pairs with valid outcomes in both conditions.

FRAMES begins from the archived eligible pool and a planned larger run, then uses the recorded budget reduction to 50 questions. The retained plan anchors already started questions and applies the recorded selection within the remaining strata. It is a budget-limited sample, not a new assertion of an outcome-independent simple random sample of the full benchmark. BASE has 49 completed answers and one provider-blocked run; ORG-DESC has 47 completed and three provider-blocked runs, with one question blocked in both. The 47 common valid pairs give 34 and 38 correct answers. Provider-blocked records, including content-policy failures, remain blocked rather than incorrect. The sampling script, launch plan, manifest, and eligible-pool provenance are linked in the evidence ledger.

TalkHier's sample is seed-42 selection of 100 questions from 1,299 official MMLU-Pro test Physics questions. The frozen subset manifest SHA256 is 3a74f4669898b1e7521dae29e6cbc154857ed2ea447acf4d800da522ce0984c6. Each arm completes 97 questions: BASE has two final errors and one timeout; ORG-DESC has one final error and two timeouts. These occur on six distinct questions, leaving 94 common valid pairs. Marginal completed-answer accuracies are 91/97 and 93/97; the primary paired comparison is instead 88/94 and 90/94. Correct-answer yield over the fixed 100-question plan is 91% and 93%, with non-completion still separately labeled. Original test row indices and official question_id are distinct identifiers.

SEAL-0 and FRAMES use their stored native semantic answer judgments. For SEAL-0, actual SearchMetric label mapping treats stored correct=1 as success, rather than relying on a conflicting class description. Previously flagged grading caveats for questions 5 and 13 are retained in the evidence instead of deleting those questions to obtain a preferred result. The main analysis uses the full 94-question scope. For TalkHier, valid native integer choices are mapped to the official option order A–J, then passed to the official MMLU-Pro update_result/save_summary functions pinned at f418b116db00b065c2aea046518d8fcf74d39872. Invalid or missing predictions remain unscored; the scorer's optional random-guess fallback is not invoked. This is official exact-match scoring of our sample, not a replication of the official few-shot inference protocol. No additional grader model is used.

For n=3 stored outcomes with c successes, the at-least-one quantity is pass@k = 1 − C(n−c,k)/C(n,k), while the all-success quantity is pass^k = C(c,k)/C(n,k), with zero for impossible combinations. At k=3 these reduce to indicators c≥1 and c=3, respectively. The first follows the pass@k interpretation and combinatorial estimator used in HumanEval [12]; the second follows the all-trial metric in τ-bench [13]. They measure breadth and consistency in the finite recorded panel. Historic rounds can differ in protocol, so the interpretation does not require us to claim independent, identically distributed production retries. Existing question-level intervals keep the paired outcomes together.

S3. Model Configuration and Valid Behavioral Returns

The studied executions use qwen3.7-flash. TalkHier's fixed batch uses temperature zero, enable_thinking=true, thinking_budget=8192, max_completion_tokens=16384, and no explicit max_tokens. It preserves native prompts and routes while using the recorded OpenAI-compatible model boundary and provider-native JSON-schema mapping. These interface compatibility changes are common to BASE and ORG-DESC. The pinned upstream TalkHier commit is 842a62160bcd326c5f8bb11c6cd1f3da7ced00f6 and the batch source commit is 0cddfdc3581bafb969b9d71e68dbeca1c4e6bdde. ROMA's historical matched rounds retain their recorded model budgets, including requests at 8,000, 32,000, and 64,000 token limits; they are not retroactively described as sharing TalkHier's budget or the later ROMA budget pilot. The source and effective configurations establish matching within contrasts, not a universal common budget across architectures or rounds. The archived role-level requests can also have different max_tokens values within a run. provenance/representative_observed_request_parameters.json records all 1,536 requests in 20 selected historical question traces, with observed caps of 8,000, 32,000 and 64,000. This is a representative effective-parameter check, not an equivalence test over every request in every trial. The archived native SEAL-0 answer judge uses qwen3.7-flash at temperature zero and max_tokens=1024; its exact prompt and frozen SearchMetric source are included.

For primary TalkHier process measures, a logical model return is normal if the callback records success, response finish_reason is not length, and the output contains nonempty text or valid tool calls. A tool-call response can legitimately have empty prose. Entirely empty ordinary returns, length-limited returns, and model exceptions are excluded from the primary behavioral call and usage totals. We do not recover token numbers from exception text for those totals. This follows the behavioral estimand in the main paper rather than a full billable-usage estimand. Missing usage remains unknown, not zero.

A model exception, a root-graph failure, and a terminal question failure are distinct layers. Root LangGraph chain errors with no langgraph_node in metadata define failed whole-graph attempts; duplicate propagation through nested callbacks does not create additional failed attempts. Normal work performed before such an attempt fails still belongs to recorded execution and enters the primary normal-return totals. On the full batch, root failures are 29 versus 35; GraphRecursionError accounts for 4 versus 19, AttributeError for 24 versus 14, and external timeouts for 1 versus 2. These are not final failed-question counts. The native retry loop's iterations counter is not incremented in the selected implementation; the external 3,600-second question limit supplies the ultimate bound. Both the behavior and this inherited

limitation are retained in the record.

The run starts at 16+16 task concurrency, then reallocates capacity to ORG-DESC after BASE completes, with a ORG-DESC limit of 32 and a realized peak of 29. The source, prompts, sample, and API budgets remain frozen, and the scheduler amendment is separately archived. The gateway records no 429 in this batch and no completed-request limit-wait total. Nevertheless, latency or throughput is not analyzed as a fixed-concurrency causal effect. The main paper uses returned call, token, and tool records rather than attributing delay to unmeasured provider queues.

Existing diagnostics separate successful-root work and a post-hoc pair subset without model, truncation, or root-attempt anomalies. Normal tokens within successful root graphs still increase by approximately 61.6%; the 50-question clean pair subset increases by 92.2%. These are previously computed conditional diagnostics, not replacements for the 94-pair primary mean or newly introduced sensitivity experiments. Different conditioning changes which questions remain, so these percentages are not interchangeable treatment estimates.

S4. Complete Trees and Supporting Structural Analyses

Logical tasks are reconstructed from native node_start records by task_id, with checks for a unique root, connected parent–child relationships, recorded depth, and node totals. Multiple functional phases of a ROMA task remain associated with that task; they do not create extra tree nodes. Model tool loops likewise do not add organizational task levels. Complete tree analysis covers all 544 SEAL-0 and 94 FRAMES trees in the primary paired panels, with 115 and 75 unique canonical shapes.

Sibling order in concurrent logs is not treated as semantic order. We canonicalize siblings by subtree size and recursive bracket representation, then compute ordered tree edit distance with APTED 1.0.3 [3], unit nonroot insert/delete cost, common structural labels, and protected root correspondence. This is distance between canonically ordered trees, not an optimal unordered tree edit distance. Raw edit distance captures changes in size as well as structure. It neither assigns quality to a particular shape nor measures coordination through a semantic node label.

The existing whole-tree geometry uses principal coordinates, a Lingoes correction for negative eigenvalues, and the full corrected coordinates for condition comparisons. Repeated coordinates are averaged within question and arm before the question-blocked PERMANOVA [14], implemented with the unmodified `vegan::adonis2` function (vegan 2.7.6). Two-dimensional plots are illustrations rather than the coordinate subset used for inference. The two primary raw-shape comparisons use a two-test Holm correction. SEAL-0 has global condition R^2=0.001977 and within-pair partial R^2=0.009537, with raw p=0.5308. FRAMES has global R^2=0.014836 and partial R^2=0.036764, with raw p=0.0213 and two-primary adjusted p=0.0426. The paired mean raw edit distances are 3.480 and 8.851, respectively; these have different task-size contexts and are not a common quality scale.

The supplementary morphology source also preserves normalized distances, role-enriched labels, dispersion checks, and multiple-comparison variants. These were conducted before this rewrite and are supplied as supporting analyses, not additional headline findings. The principal manuscript describes the strong mean-expansion response and the comparatively modest whole-tree evidence without treating every structural variant as an independent replication.

The depth-propensity analysis uses other BASE rounds to categorize a complete-repeat question's tendency to reach depth three, preventing the evaluated round from defining its own propensity. The category-specific utility changes do not form a progressively larger positive ORG-DESC gain. Paired raw tree distance has a Pearson relationship of 0.062 with correctness change in SEAL-0 and 0.120 in FRAMES; existing intervals are [−0.132,+0.265] and [−0.158,+0.387]. Some other features have descriptive associations, so these results do not imply that every behavior measure is unrelated to outcomes.

The feature analyses examine task-tree measures, tool and computational activity, and, for a restricted panel, external collaboration scores. Dependency-enriched variants include dependency density, longest dependency chain, local parallelism, and fan-in. Principal components describe joint variation and are computed by singular value decomposition of the standardized behavior matrix using NumPy, rather than scikit-learn PCA. PLS fits use scikit-learn 1.9.1 `PLSRegression` [16], with `scale=False` after training-set standardization, `max_iter=1000`, and `tol=1e-8`. Supervised PLS targets question-level ORG-DESC−BASE score changes with nested held-out evaluation: ten repetitions of five outer folds and four inner folds, training-only feature handling, and component selection within the training split. Negative held-out Q^2 means performance below the corresponding training-mean predictor; favorable in-sample fit is not predictive evidence. The preserved results include all fitted variants, component loadings, selection counts, and previously computed uncertainty and permutation results. They do not establish a stable question-level predictor in either ROMA setting.

Predictive assessment uses out-of-fold predictions, separating model/component selection from the outer test questions, following the cross-validation principle of Stone [15]. The recorded implementation defines predictive Q^2 relative to a fold-specific training-mean predictor:

$Q^2 = 1 - [\Sigma_b \Sigma_i (y_i - \hat{y}_{bi,OOF})^2] / [\Sigma_b \Sigma_i (y_i - \square_{b,train(i)})^2]$.

Here, y_i is the question-level ORG-DESC−BASE score change, $B = 10$ is the number of outer cross-validation repetitions, and each $\hat{y}_{bi,OOF}$ is predicted without fitting on question i. The reference $\square_{b,train(i)}$ is the mean outcome of that question's outer training fold, not the full-data mean. Numerator and denominator aggregate squared errors over all repetition–question predictions. Negative Q^2 therefore means higher held-out squared error than the training-mean reference. This explicit normalization is our recorded implementation convention; Stone is cited for predictive assessment by cross-validation, not as the source of this exact Q^2 variant. It is distinct from full-data-mean out-of-fold R^2 and from the separately stored ensemble metric that first averages predictions across repetitions. The main paper's predictive interpretation refers to `Q2_vs_train_mean`.

The typed DAG analysis separately retains parent→child decomposition edges and upstream→dependent result dependencies. Frozen question panels, reconstructed analysis-only graphs, bounded edit-distance records and summaries are included in data/supporting_typed_dag/. Some dependency changes are invisible to a parent-only tree. Its time-limited graph edit distances include exact solutions and bounded solutions; bounds are not relabeled exact or combined with tree edit distance into a claimed dependency contribution. This supporting evidence broadens the representation inspected without changing the main paper's task-tree estimand.

S5. External Collaboration Scoring: Version, Conversion, and Rubrics

We use the existing MARBLE Graph Planning and Graph Communication evaluation channels associated with MultiAgentBench [11]. The executed community fork cemde/MARBLE is pinned to 2efd80f9888419203f92e551815fe1e536a78cb6 and checked against official ulab-uiuc/MARBLE at 8d60fa17b5596b44458a52d4296061b9fc13d6f2. The rendered rubrics and native score parser are retained. A recorded brace-escaping fix permits Python formatting of literal JSON examples without changing the rendered rubric. The judge uses qwen3.7-flash at temperature zero and the native 512-token output limit. We reuse the existing scores and make no new rating calls.

The conversion uses each entire recorded recursive run. It retains the task tree and latest stored root dependency graph; every original system-prompt occurrence, including condition information; the model request/response ledger; and tool calls and observations. Failures and retries remain present. UUID relationships and event ordering are not invented, and a child run is not reduced to a reconstructed root-only conversation. Model and tool records include activity inside individual task executions, which is why the Communication channel should not be read as an isolated inter-agent communication measure.

Native scorer field	Full-run conversion
Planning.summary	Question plus run ledger
Planning.agent_profiles	All original system-message occurrences, with reversible encoding where applied
Planning.agent_tasks	Complete logical task tree and stored dependency ledger
Planning.results	Full model and tool ledgers
Communication.task	Question plus the same structural/run context
Communication.communications	Full model and tool ledgers, including observations and retries

Repeated literal text is replaced by ordered references to a flat block dictionary. The entire dictionary is supplied within the same scorer input, with no nested references. Byte-level restoration checks establish that this codec retains the original ledgers. Planning summary and task structure remain direct; profiles/results carry encoded content plus the dictionary footer. Communication task remains direct, while communications carries encoded content plus its footer. The input gate rejects records beyond 983,616 tokens rather than silently cropping them. This is a deterministic representation conversion, not a claim that encoding cannot influence a judge.

The main paper's common panel requires valid Planning and Communication scores in both arms for the same question–round pair, and includes only primary H/R1/R2 rounds. It contains 227 pairs across 86 questions. Scores are averaged over the same available rounds within each question, then questions receive equal weight. Composite is defined locally as (Planning+Communication)/2; MARBLE provides no third native composite channel. Planning alone has the same 227/86 coverage, whereas Communication alone could use 230/87. The shared panel makes the three reported means comparable. Larger legacy panels with extra N rounds, including 239/243 observations in an earlier post-hoc report, do not supply the main table.

The existing paired score intervals use 20,000 question bootstrap draws, seed 20261008, preserving every question's rounds and sampling the same questions across the three dimensions. The raw scores and corresponding request configurations were verified before reusing this frozen derivation. These intervals condition on the rated panel and the recorded judge outputs; the study does not repeatedly sample judge ratings to estimate judge stochasticity. On this common panel, both arms score five in 187/227 Planning pairs and 190/227 Communication pairs. Ceiling proximity, the rubric's focus on Graph self-coordination, the absence of scorer blinding to organizational prompts, and the full-ledger communication representation limit interpretation.

The exact executed Graph templates are included below and separately in scoring_prompts/native_graph_templates.json, with their license. They are retained as reference artifacts rather than rewritten to suit the present hypotheses.

S6. Package Contents, Provenance, and Reproduction

The supplement contains the exact two intervention texts in prompts/, normal-return TalkHier metrics and role summaries, complete primary question panels, the frozen reliability and collaboration derivations, and the previously computed tree/PCA/PLS outputs in data/. The primary SEAL-0 panel has 94 questions and 272 matched rounds; FRAMES has 50 planned questions and 47 valid pairs; TalkHier has 100 planned questions and 94 valid pairs. Failed, blocked, and timed-out records retain their states. Historical V2 labels mean ORG-DESC; ROMA archive condition labels alone do not identify the intervention.

Frozen source code and configurations are grouped by architecture. sources/roma/native/ comes from the archived FRAMES source freeze at native commit a6e3bb4f9e0694375fa627fa4b8bf8cae50592a6, before later budget changes in the live checkout. The historical SEAL-0 external runner snapshots are retained separately by source archive. sources/talkhier/ is the startup freeze for the primary Physics batch, with upstream commit 842a62160bcd326c5f8bb11c6cd1f3da7ced00f6 and runtime commit 0cddfdc3581bafb969b9d71e68dbeca1c4e6bdde. Native source commits, dirty working-tree capture, provider compatibility changes, model budgets, and run configurations must be considered together; a repository HEAD alone is insufficient to reproduce an archived run.

The source-record audit freshly verifies all 3,183 files in the retained primary reconciliation inventory, totaling 2,271,756,911 bytes, against their recorded SHA256 hashes. These original files remain unmodified. The complete raw ledgers are too large for the 25 MB submission limit and are not all included; instead the package supplies complete quantitative panels, frozen analysis outputs and code, representative input examples, and hashes of the original records. context_examples/ is illustrative context evidence, not additional primary trials. The frozen outputs permit offline numerical verification; executing the original agents requires independent API credentials, benchmark resources and path configuration.

provenance/source_manifest.json records both the original source hash and the submitted-file hash for each copied artifact. Personal filesystem locators are replaced with anonymous relative locators in copies. Such copies are explicitly marked as transformed, and they are not assigned the original byte hash. Intervention texts and scoring prompts retain their exact bytes. A legacy manifest attached the full multi-domain MARBLE template hash to a Graph-only extraction; the package records both files separately and verifies identity of the two executed prompt strings. No score changes follow from this metadata correction.

After extraction, run `python3 analysis/reproduce_tables.py` from the package root. The script needs only the Python standard library, verifies SHA256SUMS, recomputes Tables 2-4 from the retained per-question panels, checks three-attempt reliability and sample coverage, and prints machine-readable results. It uses no API and generates no new agent or grader calls. The confidence intervals are the existing frozen derivations, with their original seeds and procedures retained; they are not replaced by a new inferential analysis during packaging. Detailed historical analysis programs are in analysis/frozen/. The README describes relocation and execution requirements for the archived code.

S7. AI Assistance

We used OpenAI Codex (logged model identifier: gpt-6.1-sol) for discussions of organizational visibility and topology, comparisons of role-specific information in ROMA and TalkHier to inform the intervention, discussions of trajectory-analysis methods and existing benchmark metrics, and language and structural revision of author-provided material. The authors retain responsibility for the study design, evidence, interpretations, and final manuscript. Representative retained interactions and a brief description of their purposes are provided in provenance/retained_ai_prompts.json. These are selected records of multi-turn assistance, not a complete conversation log. The experimental agents and collaboration judge use qwen3.7-flash, separately documented in the execution configurations.

S5 reference: Planning rubric

```
[Context]
**Summary:** {summary}

**Agent Profiles:** {agent_profiles}

**Agent Tasks:** {agent_tasks}

**Results:** {results}

[System]
Evaluate the effectiveness of agent self-coordination in the Graph structure. Focus on:

- **Role Clarity:** Did agents understand their roles and responsibilities?
- **Task Alignment:** Were tasks aligned with goals?
- **Autonomy:** Did agents work independently without central oversight?

Rate on a 5-point scale:
1. **1 point**: Very poor self-coordination, with major inefficiencies.
2. **2 points**: Frequent role confusion, causing inefficiencies.
3. **3 points**: Moderate overlap or confusion in roles.
4. **4 points**: Effective with minor role clarification needed.
5. **5 points**: Clear roles, effective self-coordination.

[Question]
You MUST respond with ONLY a JSON object in this EXACT format:
{"rating": X}
where X is a number between 1 and 5.

Example valid responses:
{"rating": 4}
{"rating": 2}

DO NOT include any other text, explanation, or formatting.
```

S5 reference: Communication rubric

```
[Context]
**Task:** {task}

**Communications:** {communications}

[System]
Evaluate the communication quality between agents in the Graph structure. Focus on:

- **Information Exchange:** Was relevant information effectively transmitted?
- **Clarity:** Were intentions and messages clear?
- **Task Assistance:** Did communication help task completion?
- **Efficiency:** Was communication concise and purposeful?

Rate on a 5-point scale:
1. **1 point**: Poor communication with major failures.
2. **2 points**: Significant issues in clarity or relevance.
3. **3 points**: Adequate but required clarification.
4. **4 points**: Effective with minor improvements needed.
5. **5 points**: Clear, effective communication that maximized efficiency.

[Question]
You MUST respond with ONLY a JSON object in this EXACT format:
{"rating": X}
where X is a number between 1 and 5.

Example valid responses:
{"rating": 4}
{"rating": 2}

DO NOT include any other text, explanation, or formatting.
```

S1 reference: Exact ROMA description

The following is the exact intervention artifact. Inner fences use their original language labels.

```
[ROMA GLOBAL ORGANIZATIONAL ARCHITECTURE]

The following describes the global architecture of ROMA. It is a static description of how ROMA
processes tasks in general. It does not determine which path will be taken for the current task.

ORGANIZATIONAL FLOW
```sql
                    Current Task
                         |
                         v
                     Atomizer
                    /        \
            atomic /          \ planning needed
                  v            v
               Executor       Planner
                    |            |
                    |         Subtasks
                    |            |
                    |            v
                    |      each subtask enters
                    |       the same recursive
                    |        ROMA mechanism
                    |            |
                    |      child task results
                    |            |
                    |            v
                    |        Aggregator
                    |            |
                    \------------+------------>
                                 |
                           Task Result
                                 |
                       [optional Verifier]
                                 |
                            Final Output
```

ROLE RESPONSIBILITIES

Atomizer\
Determines whether the current task is atomic and can be executed directly, or whether it requires
decomposition and planning.

Planner\
When planning is required, decomposes the current task into smaller subtasks. Subtasks may have
dependencies on one another. Every generated subtask is processed recursively through the same ROMA
mechanism.

Executor\
Handles tasks that have been classified for direct execution. Execution may use reasoning, tools,
functions, APIs, retrieval, code, or other capabilities available to the Executor.

Aggregator\
Collects and integrates completed subtask results. Its purpose is to produce a coherent result for the
parent task rather than merely returning the raw outputs of its children.

Verifier\
When enabled, validates a candidate result against the original goal before final delivery. Verification
is optional and is not part of the recursive decomposition itself.

GLOBAL INFORMATION FLOW

Top-down:\
Tasks may be recursively decomposed into smaller subtasks.

Bottom-up:\
Results from subtasks are aggregated upward into solutions for their parent tasks.

Dependency-aware:\
When a subtask depends on the result of another subtask, the dependent work waits for the required
result before proceeding.

INTERPRETATION

ROMA is recursive rather than a mandatory linear pipeline. A task classified as atomic may proceed
directly from Atomizer to Executor. A task requiring decomposition proceeds from Atomizer to Planner,
and each generated subtask recursively enters the same mechanism. Completed child results are integrated
by Aggregator and returned upward until the root task result is produced.

This architecture description provides knowledge of the organization as a whole. It does not imply that
any particular branch, number of subtasks, recursion depth, dependency structure, or execution path will
occur for the current task.
```

S1 reference: Exact TalkHier Physics description

The following is the exact intervention artifact. Inner fences use their original language labels.

````
[TALKHIER GLOBAL ORGANIZATIONAL ARCHITECTURE]

The following describes the global architecture of TalkHier for this configured domain. It is a static
description of how the organization processes tasks in general. It does not determine which path will be
taken for the current task.

ORGANIZATIONAL FLOW
```text
                     Current Task
                          |
                          v
                 Default Supervisor ------------> FINISH
                          |
                          +<--> Answer Generator
                          |
                          +<--> Revisor
                          |
                          +<--> Evaluator Supervisor
                                      |
                                      +<--> Physics
                                      |
                                      +<--> Mathematics
```

Each two-way connection represents a supervisor's dispatch route to a direct member and that member's
return route to the supervisor. FINISH is the completion route of the main team.

A team invocation enters its supervisor first. After a direct member completes, the graph returns to
that team's supervisor before another direct member is selected. Returning from the evaluation team
passes through Evaluator Supervisor to Default Supervisor.

ROLE RESPONSIBILITIES

Default Supervisor
The root supervisor coordinates Answer Generator, Revisor, and Evaluator Supervisor. It selects among
its direct members, receives their returned outputs, and can finish the main-team execution through
FINISH.

Answer Generator
A direct member of Default Supervisor that produces a candidate answer for the supplied task and returns
it to Default Supervisor.

Revisor
A direct member of Default Supervisor that receives an answer and evaluation feedback, revises the
answer when needed, and returns the revision and any request for re-evaluation to Default Supervisor.

Evaluator Supervisor
A subordinate supervisor under Default Supervisor that coordinates its direct evaluation members,
collects their evaluation results, and reports the results and a summary to Default Supervisor.

Physics
An evaluation member under Evaluator Supervisor. Evaluates the correctness of the answer's physics
reasoning and assumptions. Returns its evaluation to Evaluator Supervisor.

Mathematics
An evaluation member under Evaluator Supervisor. Evaluates the correctness of the answer's calculations
and mathematical steps. Returns its evaluation to Evaluator Supervisor.
````

Exact TalkHier Physics description (continued)

```
GLOBAL INFORMATION FLOW

Dispatch and return:
Supervisors route work to their direct members. Answer Generator and Revisor return their outputs to
Default Supervisor. Evaluation members return their results to Evaluator Supervisor, which reports the
evaluation results and their summary to Default Supervisor.

Evaluation and revision:
Default Supervisor can send a candidate answer to Evaluator Supervisor for evaluation and can pass the
answer and evaluation feedback to Revisor. Revisor returns its revised answer and any request for
re-evaluation to Default Supervisor.

Feedback:
Default Supervisor can route a revised answer back through Evaluator Supervisor and its members for
another evaluation. This feedback passes through Default Supervisor rather than a direct connection
between Revisor and the evaluation members.

Scheduling:
Each supervisor selects one next member at a time. Direct-member invocations are serial within their
team, and the main team waits for a selected evaluation-team invocation to return. Evaluation members do
not have a fixed sibling order or direct sibling dispatch routes. This implementation has no separate
subtask dependency DAG.

Declared workflow requirements:
The native main-team prompt requires contacting Answer Generator first and contacting Revisor before
FINISH. A modified answer leads to a request for re-evaluation; when Default Supervisor requests
re-evaluation, the evaluation-supervisor prompt requires re-evaluating with all its members. These are
native prompt requirements. The graph itself does not check that these members have been visited before
permitting its return or completion routes.

INTERPRETATION

TalkHier has a main team with a nested evaluation branch. Default Supervisor can select Answer
Generator, Revisor, Evaluator Supervisor, or FINISH. Evaluator Supervisor can select one of its direct
evaluation members or return to Default Supervisor. Leaf members return through their assigned
supervisor. These relations describe the available organization and information flow.

This architecture description provides knowledge of the organization as a whole. It does not specify the
current task's concrete execution trace, selected evaluator order, number of evaluation or revision
rounds, answer content, or future execution state. The native prompt-level workflow requirements
described above remain applicable.
```

References

[1] Salaheddin Alzu'bi, Baran Nama, Arda Kaz, Anushri Eswaran, Weiyuan Chen, Sarvesh Khetan, Rishab Bala, Tu Vu, and Sewoong Oh. 2026. ROMA: Recursive Open Meta-Agent Framework for Long-Horizon Multi-Agent Systems. arXiv:2602.01848.

[2] Zhao Wang, Sota Moriyama, Wei-Yao Wang, Briti Gangopadhyay, and Shingo Takamatsu. 2025. Talk Structurally, Act Hierarchically: A Collaborative Framework for LLM Multi-Agent Systems. arXiv:2502.11098.

[3] Mateusz Pawlik and Nikolaus Augsten. 2016. Tree edit distance: Robust and memory-efficient. Information Systems 56, 157–173. DOI:10.1016/j.is.2015.08.004.

[4] Thinh Pham, Nguyen Nguyen, Pratibha Zunjare, Weiyuan Chen, Yu-Min Tseng, and Tu Vu. 2026. SealQA: Raising the Bar for Reasoning in Search-Augmented Language Models. ICLR 2026. arXiv:2506.01062.

[5] Satyapriya Krishna, Kalpesh Krishna, Anhad Mohananey, Steven Schwarcz, Adam Stambler, Shyam Upadhyay, and Manaal Faruqui. 2025. Fact, Fetch, and Reason: A Unified Evaluation of Retrieval-Augmented Generation. NAACL 2025. arXiv:2409.12941.

[6] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. NeurIPS 2024 Datasets and Benchmarks. arXiv:2406.01574.

[7] Xudong Guo, Kaixuan Huang, Jiale Liu, Wenhui Fan, Natalia Vélez, Qingyun Wu, Huazheng Wang, Thomas L. Griffiths, and Mengdi Wang. 2026. Embodied LLM Agents Learn to Cooperate in Organized Teams. IEEE Transactions on Computational Social Systems 13, 2, 2514–2530. DOI:10.1109/TCSS.2025.3637527. 2024 preprint: arXiv:2403.12482v2.

[8] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. Vicinagearth 1, Article 9. DOI:10.1007/s44336-024-00009-2.

[9] Yiru Wang, Xinyue Shen, Yaohui Han, Michael Backes, Pin-Yu Chen, and Tsung-Yi Ho. 2026. OrgAgent: Organize Your Multi-Agent System like a Company. arXiv:2604.01020.

[10] Chao Fei, Hongcheng Guo, and Yanghua Xiao. 2026. When Agents Evolve, Institutions Follow. arXiv:2604.27691.

[11] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. 2025. MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents. ACL 2025, Volume 1: Long Papers, 8580–8622. DOI:10.18653/v1/2025.acl-long.421.

[12] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374v2.

[13] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045v1.

[14] Marti J. Anderson. 2001. A new method for non-parametric multivariate analysis of variance. Austral Ecology 26, 1, 32–46. DOI:10.1111/j.1442-9993.2001.01070.pp.x.

[15] M. Stone. 1974. Cross-Validatory Choice and Assessment of Statistical Predictions. Journal of the Royal Statistical Society, Series B (Methodological) 36, 2, 111–133. DOI:10.1111/j.2517-6161.1974.tb00994.x.

[16] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. 2011. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 85, 2825–2830. JMLR.